

REFERENCE IMPLEMENTATION · FORENSIC EVIDENCE

Evidence Anchor

Public-key-verifiable inclusion proofs for OCSF audit events

Prepared for · OCSF data-storage-patterns discussion

Date · June 23, 2026

Scope · Public-safe: checkpoint format + offline verifier (as shipped in Case File bundles)

Integrity verification that reads no storage at all.

Relevance to the storage debate: verification reads *no database and no storage layout* — only an exported signed checkpoint and a proof. So how you store OCSF events (one wide table vs. per-class / per-source tables) is **orthogonal** to how you prove them. Choose storage purely on query performance.

1 · The problem it solves

Our OCSF audit events form a per-org, append-only **hash chain**: each event carries a per-event hash (`entry_hash`) over its canonical record and the prior event's hash. That gives tamper-evidence, but independent verification of a single event is awkward — proving one event means re-walking the chain from a trusted tip (**$O(N)$**), and the chain hash is **keyed** (secret-dependent), so the only party who can recompute it is one you'd also trust to forge it.

2 · The anchor (design in one paragraph)

A scheduled worker batches the per-event `entry_hash` values and builds a **Merkle tree (RFC 6962)**, then signs the **root once** with **ECDSA-P256** in a **DSSE** envelope; the public key is published via **JWKS**. A signed checkpoint commits an entire batch; any single event gets an **$O(\log N)$ inclusion proof** verifiable with **only SHA-256 + the public key** — no database, no shared secret. It is a verification layer on top of the chain.

```
events --> entry_hash[] --> Merkle tree (RFC 6962) --> root
                                     | sign once (ECDSA-P256, DSSE, JCS)
                                     v
                               signed checkpoint --> JWKS public key

verify 1 event:  SHA-256(proof) + ECDSA(pubkey) = committed?  ( $O(\log N)$ , offline)
```

The signed checkpoint and the proof

3 · Signed checkpoint (MerkleCheckpointV1)

Payload is JSON, canonicalized with **RFC 8785 (JCS)**, wrapped in a **DSSE** envelope. Signature is over PAE(payloadType, payload) (length-prefixed domain separation).

Field	Meaning
schema_version	1 — verifiers reject unknown versions
org_id	tenant whose chain the batch was drawn from (per-org root)
tree_size	number of leaves (events) under the root
merkle_root	SHA-256 hex of the RFC 6962 root over the batch's entry_hashes
first/last_audit_id	inclusive id range of the committed batch
signed_at	RFC 3339 (Z) signing time
signer_key_id	KMS key-version resource path; pins the key across rotations

checkpoints.json entry:

```
{
  "merkle_root": "<sha256-hex>",
  "envelope": {
    "payloadType": "application/vnd.ai-identity.anchor-checkpoint+json",
    "payload": "<base64(JCS-canonical checkpoint JSON)>",
    "signatures": [{ "keyid": "<signer_key_id>",
                     "sig": "<base64(DER ECDSA-P256)>" }]
  }
}
```

4 · inclusion-proofs.json

```
{
  "proofs": [
    { "audit_id": 84213,
      "entry_hash": "<sha256-hex>",      // the leaf value
      "index": 17,                      // position in the tree
      "tree_size": 256,
      "merkle_root": "<sha256-hex>",      // which checkpoint it proves against
      "proof": ["<sha256-hex>", "..."]   // RFC 6962 audit path, ~log2(N) hashes
    }
  ],
  "pending": [ /* exported events not yet anchored to a checkpoint */ ]
}
```

A multi-class incident is simply **N entries in proofs** — one $O(\log N)$ inclusion proof per event. pending makes coverage explicit for events newer than the last checkpoint.

What an outside verifier actually runs

5 · Bundle layout

```
evidence-anchor/
  checkpoints.json      # signed checkpoints (DSSE envelopes)
  inclusion-proofs.json # per-event proofs + pending list
```

6 · Offline verification (RFC 6962)

```
leaf_hash(d)    = SHA-256(0x00 || d)      # d = bytes.fromhex(entry_hash)
node_hash(l, r) = SHA-256(0x01 || l || r)
```

Two offline steps: **(1)** verify the DSSE/ECDSA-P256 checkpoint signature with the JWKS public key and bind the signed root to the proof's root; **(2)** run the RFC 6962 §2.1.1 audit-path check — fold `leaf_hash(entry_hash)` up through the proof and confirm it equals `merkle_root`. $O(\log N)$, SHA-256 only.

The reference verifier ships inside every bundle (Python stdlib + cryptography; no network, no DB, no secret):

```
ai_identity_verify inclusion-proof \
  --checkpoints evidence-anchor/checkpoints.json \
  --proofs      evidence-anchor/inclusion-proofs.json \
  --jwks        ai-identity-public-keys.json      # or --pubkey signer.pem
# -> INCLUSION VERIFIED - N event(s) provably committed.
```

7 · Why it matters for storage patterns

The verification path touches **no storage** — not the table layout, not an index, not a join. So single-wide-table vs. per-class/per-source is independent of integrity verification; splitting tables for query performance does not weaken proof-of-custody. It does **not** make multi-table queries faster — it's orthogonal — but it removes integrity verification as a *constraint* on the storage decision.

8 · Scope — what it does and doesn't claim

- **Does:** prove inclusion + integrity of each event (committed under a signed root, untampered), independently, with a public key, $O(\log N)$ per event.
- **Doesn't:** by itself attest the causal *ordering* between events — that linkage is the underlying hash chain. A multi-class incident is verified as a set of independent inclusion proofs.
- Recent events are pending until the next checkpoint (anchoring runs on a cadence, not synchronously).
- Roots are **per-org** (tenant isolation). The anchor sits on top of the keyed chain and needs none of its secret to verify.

References: RFC 6962 (Certificate Transparency) · RFC 8785 (JCS) · DSSE · OCSF.